Opticos User Manual

Written by David Green

Introduction

Hello and welcome to the Opticos user manual! In this manual we will be going over what Opticos is, the different features that the program offers, and how to run each individual feature.

What is Opticos?

Opticos is educational math tool with the goal of visually demonstrating complex math topics using precise computer animations. The name Opticos come from the greek word "optikos", which means visual. When we say that Opticos uses animations, we don't mean the kind used as children's entertainment. We mean the type of animations used in computer simulations, in which objects are moved in a precise way to visualize complex ideas.

Furthermore, the motivation behind Opticos is that there is lack of modern accesible tools to visualize higher level math concepts. The computer has come such a long way and yet when it comes to math education, we still rely almost exclusively on messy hand drawings on white boards or chalkboards to visualize concepts. With Opticos, we aim to provide an additional supplementary tool to help visually explain math concepts in a clearer and more complete way. We want Opticos to be a useful resource for students who want to learn and a useful resource for instructors who want to help better explain concepts.

The Two Features of Opticos

Opticos has two different features, the **Interactive Animator** and **the Opticos Textbook**. The interactive animator is an tool that ask you to input a function and a set of other input values which are then used to create an animation demonstrating a particular concept. The Opticos Textbook is a mini textbook with standard text explanations of math concepts but with a little twist. Accompanying the text explanation are pre-rendered animations to help support said explanations. Let's talk more in detail about each of these two features.

The Interactive Animator

What is the interactive animator?

Suppose you're a student who wants to a see a concept demonstrated in action on a function that appears in a homework problem, or you're a professor who wants to use a very specific function to visually demonstrate an idea to your class. Drawing it out by hand can sometimes be useful but it can often be messy and inefficient. With the interactive animator, you can input a function along with a set of additional parameters to create a precise animation that demonstrates a particular concept using the function and parameters you gave.

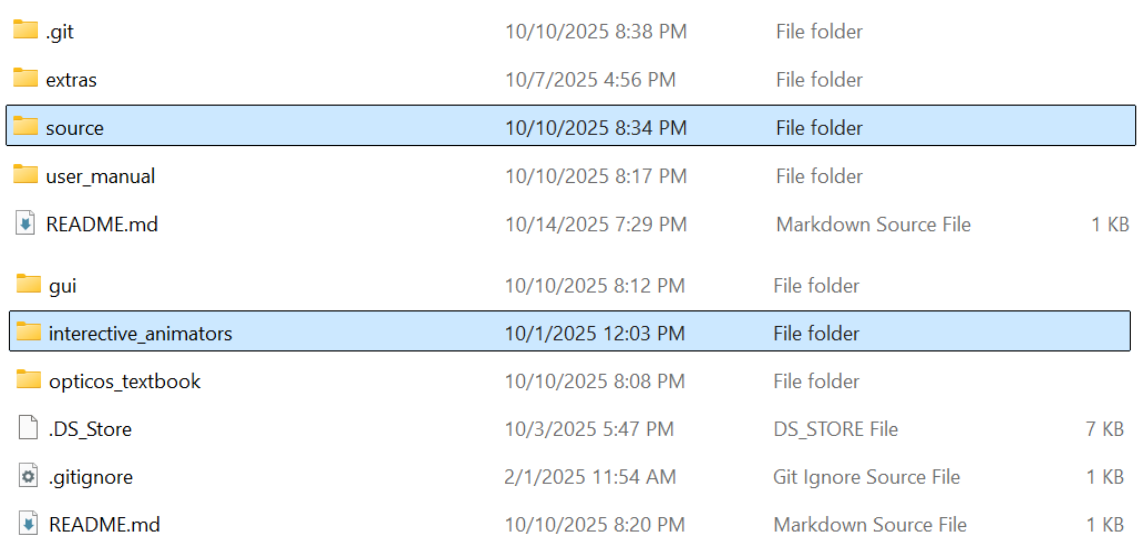
Types of Interactive Animators Available

As of now, we have two interactive animators with one currently in development. Currently available is the Limit Animator and the Integral Animator. We're currently working on a Derivative Animator with the goal of a prototype version being available by the end of 2025.

How to run the Interactive Animator

Ideally we would like all the features of Opticos to be runnable through a GUI that can be opened by a single executable file but we're not currently there yet. As of now, the interactive animator can only be run by directly running the source code for each respective animator.

In the root of the repository, open the source directory and then a directory called interactive_animators



.git	10/10/2025 8:38 PM	File folder	
extras	10/7/2025 4:56 PM	File folder	
source	10/10/2025 8:34 PM	File folder	
user_manual	10/10/2025 8:17 PM	File folder	
README.md	10/14/2025 7:29 PM	Markdown Source File	1 KB
gui	10/10/2025 8:12 PM	File folder	
interective_animators	10/1/2025 12:03 PM	File folder	
opticos_textbook	10/10/2025 8:08 PM	File folder	
.DS_Store	10/3/2025 5:47 PM	DS_STORE File	7 KB
.gitignore	2/1/2025 11:54 AM	Git Ignore Source File	1 KB
README.md	10/10/2025 8:20 PM	Markdown Source File	1 KB

From here, you will see a repository for each of the different interactive animators.

derivative_animator	10/10/2025 8:10 PM	File folder
integral_animator	10/10/2025 8:10 PM	File folder
limit_animator	10/10/2025 8:11 PM	File folder

If you click on one of these, for example, the one for the limit animator, you will see two python files. One is of the form render_<concept>_animation.py and the other is of the form user_input_<concept>.py. Open the one of the form user_input_<concept>.py

__pycache__	10/20/2025 11:26 PM	File folder	
media	2/14/2025 5:32 PM	File folder	
README.md	10/10/2025 8:12 PM	Markdown Source File	1 KB
render_limit_animation.py	9/30/2025 5:02 PM	Python Source File	8 KB
user_input_limit.py	10/20/2025 11:26 PM	Python Source File	2 KB

From there, run the python file through either your preferred IDE or the terminal. Doing so will prompt you to enter a set of input parameters for the animation.

```
PS C:\Users\David> python -u "c:\Users\David\Opticos\source\interactive_animators\limit_animator\user_input_limit.py"
Enter a function: f(x) = 2x^2 + 4x - 3
Enter value you want x to approach: 2
Do you want to approach it from the left or from the right? (-1 for left, 1 for right): -1
```

It will take a little bit of time for the animation to render so wait a bit for the program to do its thing. Once the animation is done being rendered it will save the animation as an mp4 file and and tell you the location of the mp4.

```
INFO     Combining to Movie file.
INFO
         File ready at 'C:\Users\David\media\videos\1080p60\limit_animation.mp4'

INFO     Rendered limit_animation
         Played 3 animations
```

Note that at the bottom it may say "played n animations". In this case it says "played 3 animations" That does not mean that 3 separate animation files were made, that just refers to the number of sub animations generated in the back end that make up the entire mp4 file, so you can just ignore it.

The Limit Animator

The first available interactive animator is the limit animator. The limit animator takes three input paramters:

1. The function
2. The value you want x to approach
3. The side you want to approach that point from

As of now, the only functions that are compatible with the limit animator are polynomials, which are functions of the form $a_n x^n + a_{n-1} x^{n-1} + ... + a_2 x^2 + a_1 x + a_0$. Also note that the polynomial must be in expanded form so although $(x+2)^2$ is technically a polynomial, it must be written as x^2+4x+4.

For the approach value, it can be any real number, there's no restrictions. Note that extremely large numbers may significantly increase the rendering time for the animation.

For the side you want the approach the point from, you can either enter -1 to approach it from the left or 1 to approach it from the right.

Now what exactly does the actual animation look like. First, the function is plotted out on a graph. Then a point moves along the graph, approaching the value you entered as the second parameter. On the side of the screen is a box that updates the x and y coordinates of the point every frame. The point then stops just before the approach value and the value of the limit is displayed on the screen.

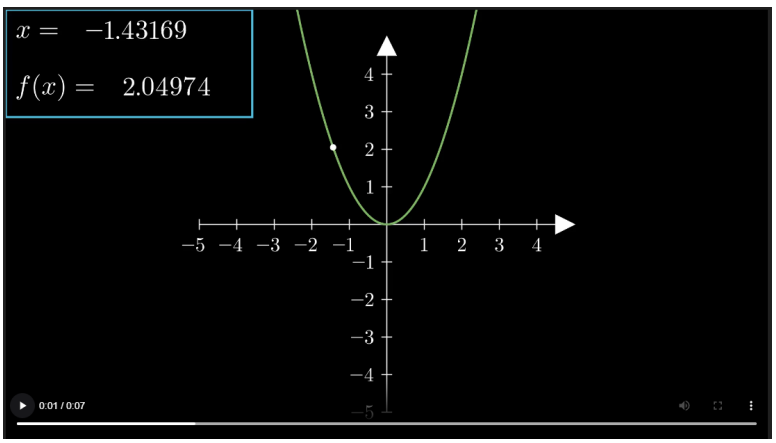
On the next pages are three example inputs for the limit animator.

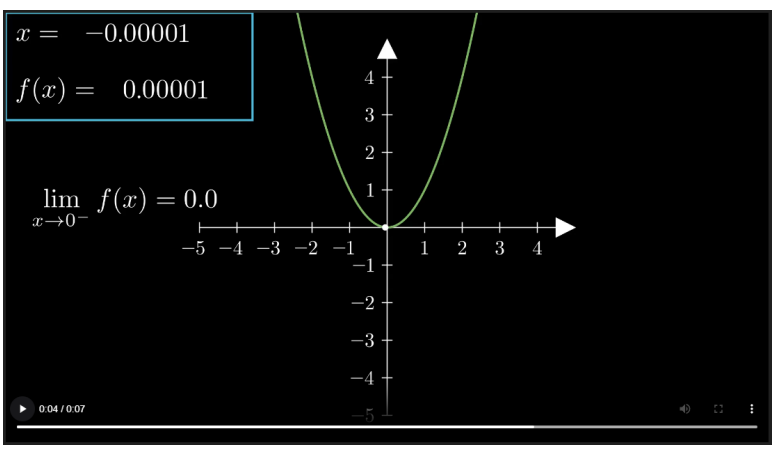
Example 1

Suppose we enter the following input values:

```
Enter a function: f(x) = x^2
Enter value you want x to approach: 0
Do you want to approach it from the left or from the right? (-1 for left,
1 for right): -1
```

This means that we want to make an animation for $\lim_{x \to 0^-} x^2$. Here's some images of what the output animation will look like.



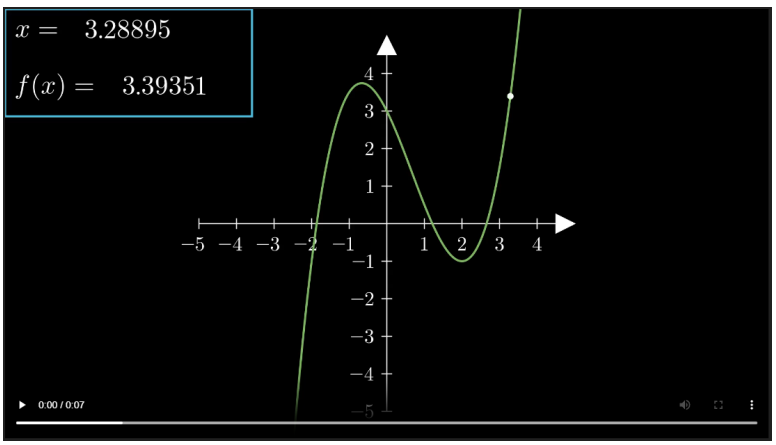


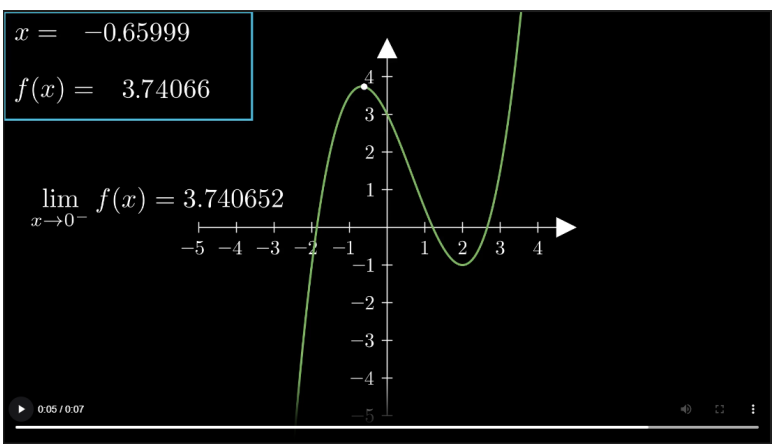
Example 2

Suppose we enter the following input values:

```
Enter a function: f(x) = 0.5x^3 - x^2 - 2x + 3
Enter value you want x to approach: -0.66
Do you want to approach it from the left or from the right? (-1 for left,
1 for right): 1
```

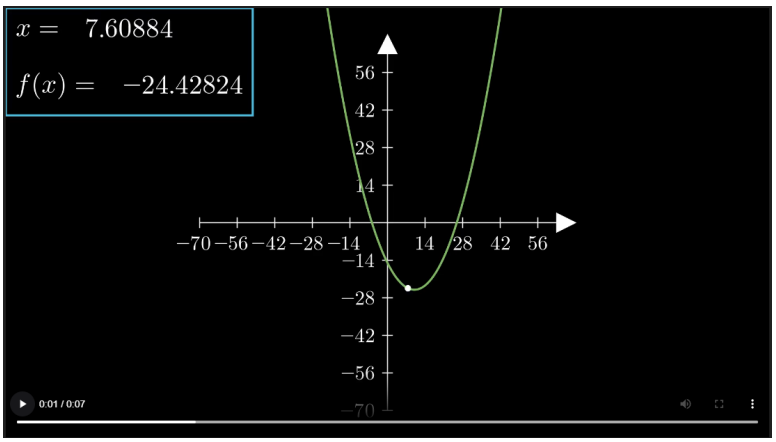
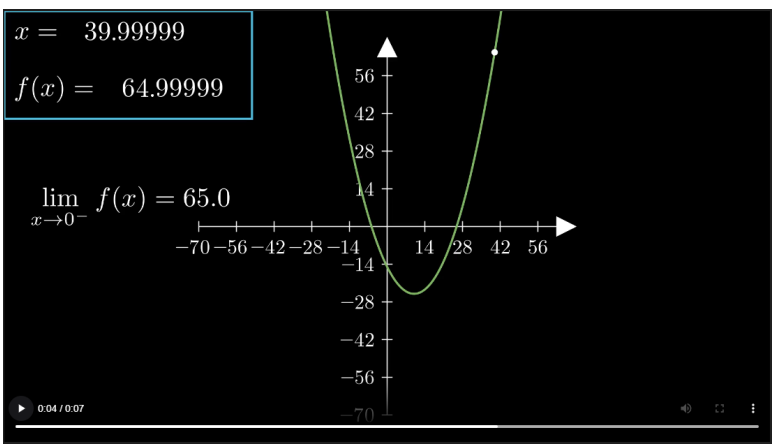
This means that we want to make an animation for $\lim_{x \to 0.66^+} 0.5x^3 - x^2 - 2x + 3$. Here's some images of what the output animation will look like.





Example 3

Suppose we enter the following input values:

```
Enter a function: f(x) = 0.1x^2 - 2x - 15
Enter value you want x to approach: 40
Do you want to approach it from the left or from the right? (-1 for left,
1 for right): -1
```

This means that we want to make an animation for $\lim_{x \to 40^-} 0.1x^2 - 2x - 15$. Here's some images of what the output animation will look like.